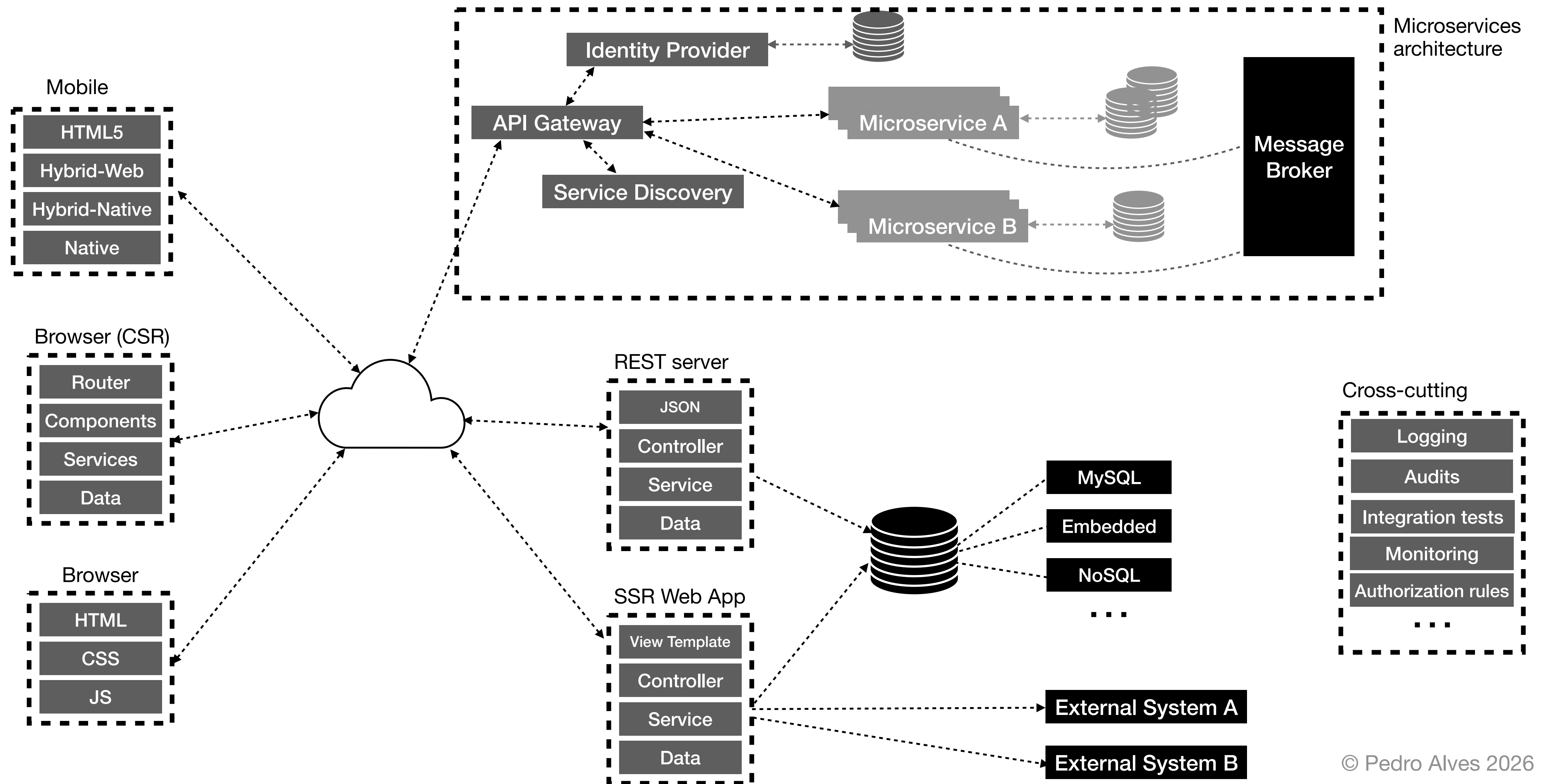


Internet Applications Design and Implementation

Week 12 - Recap & Trends.

The big picture



SSR vs CSR

Would you implement a CRUD
in SSR or CSR?

New Event

Chess Club

Event Name *

Date *



Location *

Event Type *



Description



Create Event

```
<div class="container">
  <div class="row justify-content-center">
    <div class="col-md-6">

      <h1>New Event</h1>
      <h2 class="text-muted fs-5 fw-normal">Chess Club</h2>

      <form action="/clubs/1/events"
        method="post">

        <div class="mb-3">
          <label for="name" class="form-label">Event Name <span class="text-danger">*</span></label>
          <input type="text" id="name" class="form-control"
            placeholder="Enter event name" name="name" value=""/>

        </div>

        <div class="mb-3">
          <label for="date" class="form-label">Date <span class="text-danger">*</span></label>
          <input type="date" id="date" name="date" class="form-control"
            value=""/>

        </div>

        <div class="mb-3">
          <label for="location" class="form-label">Location <span class="text-danger">*</span></label>
          <input type="text" id="location" class="form-control"
            placeholder="Enter location" name="location" value=""/>

        </div>

        <div class="mb-3">
          <label for="type" class="form-label">Event Type <span class="text-danger">*</span></label>
          <select id="type" class="form-select" name="type">
            <option value="">Select type...</option>
            <option value="WORKSHOP">WORKSHOP</option>
            ...
          </select>

        </div>

        <div class="mb-3">
          <label for="description" class="form-label">Description</label>
          <textarea id="description" class="form-control"
            rows="3" placeholder="Optional description" name="description"></textarea>

        </div>

        <button type="submit" class="btn btn-primary">Create Event</button>

      </form>

    </div>
  </div>
</div>
```

SSR

58 lines

New Event

Chess Club

Event Name *

Enter event name

Date *

dd/mm/yyyy

Location *

Enter location

Event Type *

Select type...

Description

Optional description

Create Event

```
import React, { FormEvent, useState } from "react";

const eventTypes = [
  "WORKSHOP", "TALK", "COMPETITION", "SOCIAL",
  "MEETING", "OTHER",
] as const;

type EventType = "" | (typeof eventTypes)[number];

type EventFormData = {
  name: string;
  date: string;
  location: string;
  type: EventType;
  description: string;
};

export default function NewEventForm() {
  const [formData, setFormData] = useState<EventFormData>({
    name: "",
    date: "",
    location: "",
    type: "",
    description: "",
  });
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [errorMessage, setErrorMessage] = useState("");

  const handleChange = (
    event: React.ChangeEvent<HTMLInputElement | HTMLSelectElement | HTMLTextAreaElement>
  ) => {
    const { name, value } = event.target;

    setFormData((current) => ({
      ...current,
      [name]: value,
    }));
  };

  const handleSubmit = async (event: FormEvent<HTMLFormElement>) => {
    event.preventDefault();
    setIsSubmitting(true);
    setErrorMessage("");

    try {
      const response = await fetch("/clubs/1/events", {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify(formData),
      });

      if (!response.ok) {
        throw new Error("Failed to create event. Please check the form and try again.");
      }

      window.location.href = response.url || "/clubs/1/events";
    } catch (error) {
      setErrorMessage(error instanceof Error ? error.message : "Something went wrong.");
    } finally {
      setIsSubmitting(false);
    }
  };

  return (
    <div className="container">
      <div className="row justify-content-center">
        <div className="col-md-6">
          <h1>New Event</h1>
          <h2 className="text-muted fs-5 fw-normal">Chess Club</h2>

          <form onSubmit={handleSubmit}>

            <div className="mb-3">
              <label htmlFor="name" className="form-label">Event Name <span className="text-danger">*</span></label>
              <input type="text" id="name" className="form-control" placeholder="Enter event name"
                name="name" value={formData.name} onChange={handleChange}/>
            </div>

            <div className="mb-3">
              <label htmlFor="date" className="form-label">Date <span className="text-danger">*</span></label>
              <input type="date" id="date" name="date" className="form-control"
                value={formData.date} onChange={handleChange}/>
            </div>

            <div className="mb-3">
              <label htmlFor="location" className="form-label">Location <span className="text-danger">*</span></label>
              <input type="text" id="location" className="form-control" placeholder="Enter location"
                name="location" value={formData.location} onChange={handleChange}/>
            </div>

            <div className="mb-3">
              <label htmlFor="type" className="form-label">Event Type <span className="text-danger">*</span></label>
              <select id="type" className="form-select" name="type"
                value={formData.type} onChange={handleChange}>
                <option value="">Select type...</option>
                {eventTypes.map((type) => (
                  <option key={type} value={type}>
                    {type}
                  </option>
                ))}
              </select>
            </div>

            <div className="mb-3">
              <label htmlFor="description" className="form-label">Description</label>
              <textarea id="description" className="form-control" rows={3} placeholder="Optional description"
                name="description" value={formData.description} onChange={handleChange}/>
            </div>

            {errorMessage && <div className="alert alert-danger">{errorMessage}</div>}

            <button type="submit" className="btn btn-primary" disabled={isSubmitting}>
              {isSubmitting ? "Creating..." : "Create Event"}
            </button>
          </form>
        </div>
      </div>
    </div>
  );
}
```

CSR

122 lines
(+ 50K lines in React libraries)

New Event

Chess Club

Event Name *

Enter event name

Date *

dd/mm/yyyy

Location *

Enter location

Event Type *

Select type...

Description

Optional description

Create Event

Case study

Client-side navigation in Google Forms

Perguntas Respostas 26 Definições

26 respostas [Ver no Sheets](#)

Resumo Pergunta Individual

< 9 de 26 >

Não é possível editar as respostas

IADI 2026 - Course Feedback Survey

Thank you for taking the time to complete this survey. Your feedback is valuable!

This survey is anonymous, and your responses will be used only for academic and course improvement purposes. Please answer the questions honestly based on your experience throughout the course.

* Indica uma pergunta obrigatória

How do you feel about what you have learned in this course, so far? *

1 2 3 4 5

Very dissatisfied ☐ ☐ ☐ ☒ ☐ Very satisfied

1. Initially, Google returns the HTML with all the elements of the response filled in
2. On each click to get a new response, Google calls an endpoint (via fetch) to get that single response in JSON

`getsingleresponse?sid=7272151e152caffd&vc=0&c=0&w=...V4peqi`

3. It replaces (rehydrates) specific HTML elements with JSON data

Consequence: the server must be prepared to return a full HTML (SSR) or JSON data

Client-side navigation in Google Forms

The screenshot shows the Google Forms interface for a survey titled "IADI 2026 - Course Feedback Survey". At the top, there are tabs for "Perguntas", "Respostas" (with a count of 26), and "Definições". Below the tabs, the "Respostas" section is active, showing "26 respostas" and a "Ver no Sheets" button. The survey content includes a thank you message, a statement about the survey being anonymous, and a rating question: "How do you feel about what you have learned in this course, so far? *". The rating scale ranges from 1 (Very dissatisfied) to 5 (Very satisfied), with the 4th option selected. A legend at the bottom indicates that a red asterisk (*) denotes a mandatory question.

Alternative 1: Each click gets a new page from the server (Server-side navigation)

Alternative 2: Each click triggers a HTMX (swap) of the form
The server either renders the full page or a fragment

SPAs

“

I think SPAs, as developed over the last decade, have mostly been a mistake. In my experience they take longer to build, have more bugs and provide worse performance than a server-side, multi-page alternative implementation.

Obviously if you are building Figma or VS Code then SPAs are the right way to go. But most web applications are not Figma, and don't need to be!

”

Simon Willison (co-creator of Django Web Framework)

<https://simonwillison.net/2022/May/4/datasette-lite/>

Microservices pain points

After implementing a *very simple* application based on microservices, what were the biggest pain points?

- even with logs, it's difficult to understand what's going on
 - several logs to look at
 - difficult to trace the execution
 - debugging is harder
- duplicated code across microservices
 - DTOs, logging handlers, security services (e.g, JWT validation), ...
- high startup overhead for local development environments
 - several mysql instances, eureka, api gateway, all microservices, webapp, ...

Microservices

“

In other words, Microservices Architectures fits the organizational shape of very large corporations. So far so good!

(...)

The patterns that make sense for organizations orders of magnitude larger than yours, are often the exact opposite ones that'll make sense for you. It's the essence of cargo culting. If I dance like these behemoths, surely I too will grow into one. I'm sorry, but that's just not how the tango goes.

”

David Heinemeyer Hanson (creator of Ruby on Rails)

<https://signalvnoise.com/svn3/the-majestic-monolith/>

Microservices

“

And my advice has been, again, for over a decade now, to start with a monolith first.(...) I have to be convinced that now is the time to split off a service. Running and operating a more distributed system incurs costs.

”

Sam Newman (author of Building Microservices)

<https://www.infoq.com/podcasts/microservices-benefits-supersede-caveats/>

Case study - Stackoverflow (2011)



- 95 Million Page Views a Month
- 800 HTTP requests a second
- 16 Million Users

- Windows Server + ASP.NET Application (C#) + SQL Server database
- Monolith, Server-side rendering
- No cloud provider - several (load-balanced) servers in 2 data centers
- “Stack Overflow's database is almost completely in RAM”

<https://highscalability.com/stack-overflow-architecture/>

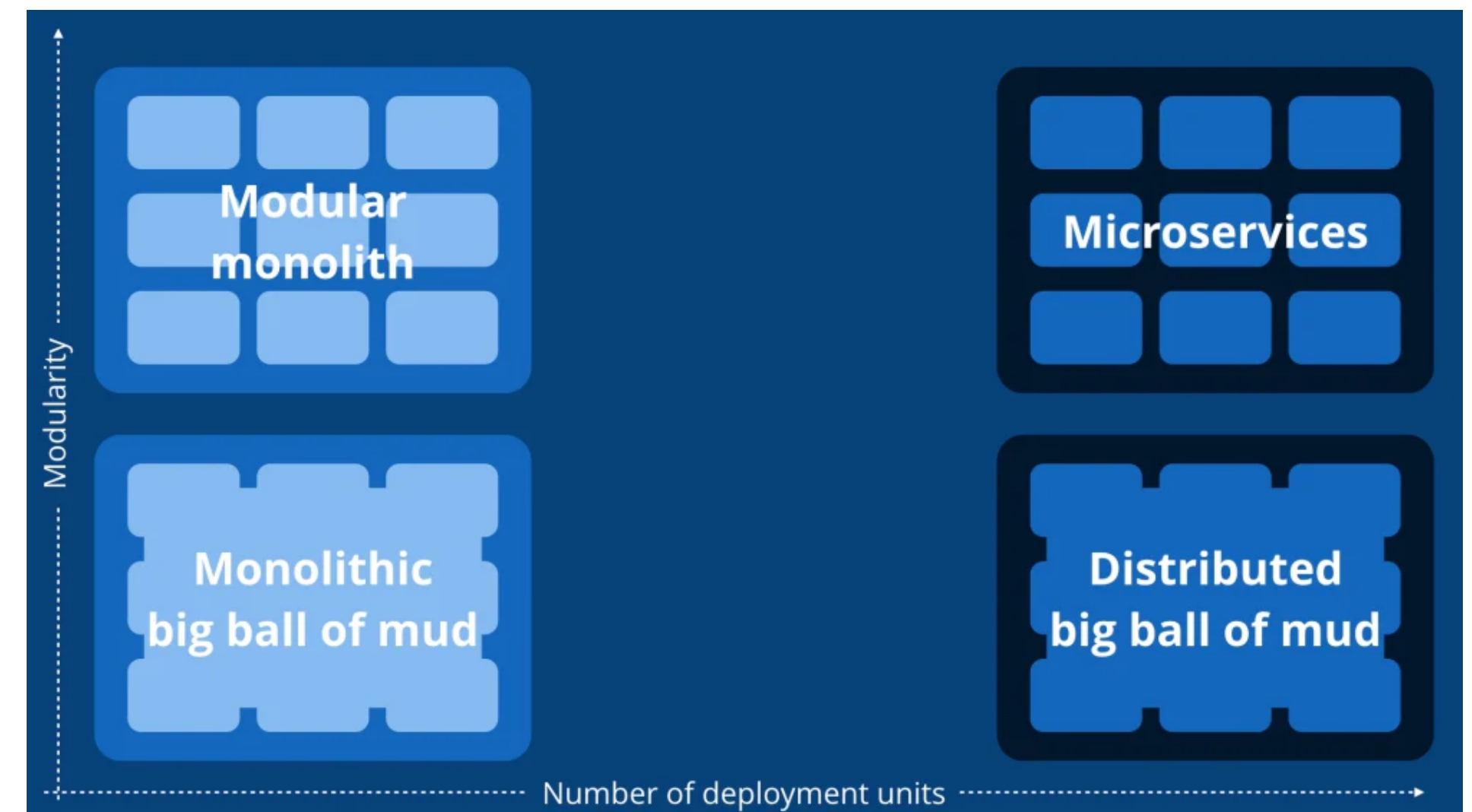
<https://highscalability.com/stack-overflow-architecture-update-now-at-95-million-page-vi/>

Case study - Shopify (2019)

- powers ~6 million live stores globally
- uses a “modular monolith” instead of microservices

“We wanted a solution that increased modularity without increasing the number of deployment units, allowing us to get the advantages of both monoliths and microservices without so many of the downsides.”

- developed in Ruby on Rails
- most shopify stores use Server-side rendering



Modular monoliths

Single codebase with:

- well-defined business modules
- strict boundaries enforced at compile time
- explicit interfaces between modules (direct calls)

Advantages:

- faster/easier development and debugging (e.g., refactoring across modules is possible)
- no network latency between services
- easier local development & testing
- lower infrastructure/cloud costs
- simpler deployments and observability

Hard-Earned Engineering Lessons

from my personal “war stories”

- Architectural decisions are political/business, not necessarily technical
- NoSQL is often just a glorified file system
- Run away from event buses (relational databases and cronjobs will do the job just fine)
- The real bottleneck is not where you think it is (but always suspect of the database)
- Write code for the next developer, not for yourself
- YAGNI (You are not going to need it)

Choose boring technology

“ Hey I just heard about this new graph database. I think that we should try it out.

- Someone who is about to have a bad time

consider how you would solve your immediate problem
without adding anything new

total cost = $\sum(\text{maintenance cost}) - \sum(\text{velocity benefit})$

Course Retrospective

- History of web applications 
- Modularization, coupling, IoC/DI patterns 
- Software architecture: two-tier, three-tier, services, microservices  (revised to focus on modularization and dependencies)
- RESTful applications 
- Web Frameworks  (specific focus on Spring to help with the project)
- Server-side rendering: View templates, Forms, Navigation  (and much more...)
- Persistence 
- Security, Authentication and Authorization 
- Traditional web application architecture  (bad name, we actually talked about dynamic frontends without CRS and automated tests)
- Microservices architecture 
- Client-side rendering: MPAs and SPAs; Reactive component-based frameworks  (with a strong focus on React)
- Mobile application development 

Take aways from this course

- modularity and loose coupling are your goals, but don't forget developer productivity
- for every decision:
 - enumerate all possible solutions
 - research their pros and cons (if you can't find cons, you haven't researched enough)
 - always consider technical, business AND organizational factors
- disclaimer: you won't become an architect by learning stuff from a course
 - work/suffer first through several projects as a developer
 - always try to understand how your (tiny) piece fits into the whole system

Let's keep in touch

<https://www.linkedin.com/in/palves/>